

go - golang - programming - concurrency Oct 23 - Written By Noah Parker

Understanding Golang's Atomic Package and Mutexes

Go is a modern programming language designed to simplify the development of scalable and concurrent software. Two of its major concurrency primitives are the atomic package and mutexes. In this post, we'll delve into both to understand their usage and when one might be preferable over the other.

Concurrency in Go

Concurrency is a core part of the Go programming language. This is facilitated by goroutines, which can be thought of as lightweight threads managed by the Go runtime. However, with great power comes great responsibility. When multiple goroutines access shared data, we can run into race conditions, which can cause unpredictable results. Hence, Go provides synchronization tools like the atomic package and mutexes.

The Atomic Package

The `sync/atomic` package provides low-level atomic memory primitives which can be used to modify values without the fear of race conditions.

Key features of the atomic package:

- **Atomic Operations:** These functions allow operations on integers and pointers to be performed atomically. For example, `atomic.AddInt64` adds an integer to an `int64` value atomically.
- **Memory Ordering Guarantees:** Atomic operations are not just about ensuring a single operation is performed without interruption, but they also ensure proper memory ordering. This means that operations before the atomic operation in program order are guaranteed

to be observed before the atomic operation, and operations after the atomic operation are observed after.

Usage Example:

```
var counter int32

func increment() {
    atomic.AddInt32(&counter, 1)
}
```

Mutexes

While atomic operations are powerful, they are limited to simple operations. For more complex operations, mutexes are more suitable. Mutex stands for "mutual exclusion", and it ensures that only one goroutine can access a critical section of code at a time.

Go's `sync` package provides `Mutex` and `RWMutex` (read-write mutex) types.

Usage Example:

```
var counter int
var m sync.Mutex

func increment() {
    m.Lock()
    counter++
    m.Unlock()
}
```

Atomic vs Mutex: When to Use Which?

- **Simplicity vs Complexity:** Atomic operations are suitable for simple operations like incrementing a counter. Mutexes are for more complex operations or multiple operations that need to be protected together.
- **Performance:** Atomic operations are generally faster than mutexes for their intended use-cases because they don't involve locking and blocking. However, overusing atomics, especially in non-trivial scenarios, can make code hard to understand and maintain.
- **Read-heavy operations:** If you have a situation where there are numerous reads and occasional writes, `RWMutex` can be beneficial. It allows multiple goroutines to read the data simultaneously but ensures exclusive access for writes.

Tips and Best Practices

1. **Always Unlock Mutexes:** Always ensure that a locked mutex is unlocked. A common practice is to defer the unlock just after locking.
2. **Minimize Locked Time:** Hold a mutex lock for as short a duration as possible to reduce contention.
3. **Use Atomic Where Suitable:** For simple operations, atomic functions can provide a performance benefit. However, clarity should not be sacrificed for a slight performance gain.
4. **Beware of Deadlocks:** Ensure that the order of acquiring multiple locks is consistent across goroutines to prevent deadlocks.

More Examples

Atomic Compare and Swap (CAS)

CAS is a crucial operation that checks if the current value is equal to an expected value and, if it is, sets it to a new value. It returns a boolean indicating if the swap was successful.

```
var value int32 = 42
```

```
success := atomic.CompareAndSwapInt32(&value, 42, 100)  
if success {
```

```

    fmt.Println("Value was 42, now it's 100")
} else {
    fmt.Println("Value wasn't 42, no change made")
}

```

Working with Pointers

Atomic operations aren't limited to integers. They can also work with pointers.

```

type Config struct {
    Key    string
    Value  string
}

currentConfig := &Config{"defaultKey", "defaultValue"}
newConfig := &Config{"newKey", "newValue"}

atomic.StorePointer((*unsafe.Pointer)(unsafe.Pointer(&currentConfig)),
    unsafe.Pointer(newConfig))

```

Atomic Add for Floats

Go's atomic package doesn't provide functions for floating-point numbers directly, but you can create such functions using the available atomic primitives. Here's a basic example of how to implement an atomic add for `float64`:

```

func atomicAddFloat64(val *uint64, delta float64) (new float64) {
    for {
        old := atomic.LoadUint64(val)
        new = math.Float64frombits(old) + delta
        if atomic.CompareAndSwapUint64(val, old, math.Float64bits(new)) {
            return
        }
    }
}

```

```
}  
}
```

```
var floatVal uint64  
atomicAddFloat64(&floatVal, 3.14)
```

Note: This example uses a CAS loop, which tries to update the value until it succeeds.

Concurrency in Go offers powerful tools and mechanisms to write efficient and scalable code. Both the atomic package and mutexes have their places in a Go developer's toolkit. The key is understanding when and where to use each for effective and safe concurrent programming.

go - golang - mutex - atomic

Noah Parker

◀ **Understanding Empty
Interfaces in Go**

**Manual Memory
Management Techniques
using unsafe in Go** ▶